



Complete Bazel Guide: From Scratch to Advanced with C++

A comprehensive guide to mastering Bazel build system with C++ examples

| Table of Contents

1. Introduction to Bazel
2. Installation & Setup
3. Bazel Basics
4. C++ Build Rules
5. Workspaces & Packages
6. Managing Dependencies
7. Advanced C++ Features
8. Testing with Bazel
9. Build Optimization
10. Custom Toolchains
11. Best Practices

1. Introduction to Bazel

What is Bazel?

Bazel is a powerful, open-source build and test tool developed by Google. It supports multiple languages and platforms, offering fast, reliable, and reproducible builds.

Key Features

- **Fast Builds:** Incremental builds and parallel execution
- **Reproducible:** Same inputs always produce same outputs
- **Multi-language:** C++, Java, Python, Go, and more
- **Scalable:** Handles projects from small to Google-scale
- **Remote Execution:** Distributed build capability

Note: Bazel uses a declarative build language called Starlark, which is similar to Python.

2. Installation & Setup

Linux Installation

```
# Using Bazelisk (recommended)
wget https://github.com/bazelbuild/bazelisk/releases/latest
chmod +x bazelisk-linux-amd64
sudo mv bazelisk-linux-amd64 /usr/local/bin/bazel

# Verify installation
bazel --version
```

macOS Installation

```
# Using Homebrew
brew install bazelisk

# Verify
```

```
bazel --version
```

Windows Installation

```
# Using Chocolatey
```

```
choco install bazelisk
```

```
# Or download from GitHub releases
```

```
# https://github.com/bazelbuild/bazel/releases
```

3. Bazel Basics

3.1 Workspace Structure

A Bazel workspace is a directory containing your source code and a WORKSPACE file at the root.

```
my_project/
├── WORKSPACE
├── .bazelrc
├── BUILD
├── src/
│   ├── BUILD
│   ├── main.cpp
│   └── lib/
│       ├── BUILD
│       ├── hello.h
│       └── hello.cpp
└── tests/
    ├── BUILD
    └── hello_test.cpp
```

3.2 WORKSPACE File

The WORKSPACE file defines the project root and external dependencies.

```
# WORKSPACE
workspace(name = "my_cpp_project")

# Load C++ rules
load("@bazel_tools//tools/build_defs/repo:http.bzl", "http_

# Example: Loading external dependencies
http_archive(
    name = "com_google_googletest",
    urls = ["https://github.com/google/googletest/archive/1
    strip_prefix = "googletest-release-1.12.1",
)
```

3.3 BUILD Files

BUILD files define targets (libraries, binaries, tests) in each package.

```
# BUILD file syntax
rule_type(
    name = "target_name",
    srcs = ["source_files"],
    hdrs = ["header_files"],
    deps = ["dependencies"],
    visibility = ["//visibility:public"],
)
```

4. C++ Build Rules

4.1 cc_library

Creates a C++ library that can be used by other targets.

```
# src/lib/BUILD
cc_library(
    name = "hello",
    srcs = ["hello.cpp"],
    hdrs = ["hello.h"],
    visibility = ["//visibility:public"],
)

// src/lib/hello.h
#ifndef HELLO_H
#define HELLO_H

#include <string>

namespace mylib {
    std::string getGreeting(const std::string& name);
    void printGreeting(const std::string& name);
}

#endif

// src/lib/hello.cpp
#include "src/lib/hello.h"
#include <iostream>

namespace mylib {
    std::string getGreeting(const std::string& name) {
        return "Hello, " + name + "!";
    }

    void printGreeting(const std::string& name) {
        std::cout << getGreeting(name) << std::endl;
    }
}
```

```
    }  
}
```

4.2 cc_binary

Creates an executable C++ program.

```
# src/BUILD  
cc_binary(  
    name = "main",  
    srcs = ["main.cpp"],  
    deps = ["//src/lib:hello"],  
)  
  
// src/main.cpp  
#include "src/lib/hello.h"  
  
int main(int argc, char** argv) {  
    std::string name = (argc > 1) ? argv[1] : "World";  
    mylib::printGreeting(name);  
    return 0;  
}
```

Building and Running:

```
# Build the binary  
bazel build //src:main  
  
# Run the binary  
bazel run //src:main  
  
# Run with arguments  
bazel run //src:main -- Alice
```

4.3 cc_test

Creates a C++ test target.

```
# tests/BUILD
cc_test(
  name = "hello_test",
  srcs = ["hello_test.cpp"],
  deps = [
    "//src/lib:hello",
    "@com_google_googletest//:gtest_main",
  ],
)

// tests/hello_test.cpp
#include "src/lib/hello.h"
#include <gtest/gtest.h>

TEST(HelloTest, BasicGreeting) {
  std::string result = mylib::getGreeting("World");
  EXPECT_EQ(result, "Hello, World!");
}

TEST(HelloTest, CustomName) {
  std::string result = mylib::getGreeting("Alice");
  EXPECT_EQ(result, "Hello, Alice!");
}
```


5. Workspaces & Packages

5.1 Label Syntax

Bazel uses labels to reference targets:

Label Format	Description	Example
<code>// package:target</code>	Absolute label	<code>//src/lib:hello</code>
<code>:target</code>	Target in current package	<code>:main</code>
<code>//package</code>	Target with same name as package	<code>//src/lib</code> → <code>//src/lib:lib</code>
<code>@workspace// package:target</code>	External workspace	<code>@com_google_googletest//:gtest</code>

5.2 Visibility

Control which packages can depend on your targets:

```
cc_library(  
    name = "internal_lib",  
    # Only visible within this package  
    visibility = ["//visibility:private"],  
)  
  
cc_library(  
    name = "public_lib",  
    # Visible to entire project  
    visibility = ["//visibility:public"],  
)  
  
cc_library(  
    name = "external_lib",  
    # Visible to entire project  
    visibility = ["//visibility:public"],  
    deps = ["@com_google_googletest//:gtest"]  
)
```

```
    name = "selective_lib",  
    # Visible to specific packages  
    visibility = [  
        "//src/apps:__pkg__",  
        "//tests:__subpackages__",  
    ],  
)
```

5.3 Package Groups

```
# Define a package group  
package_group(  
    name = "friends",  
    packages = [  
        "//src/...",  
        "//internal/...",  
    ],  
)  
  
cc_library(  
    name = "restricted_lib",  
    visibility = [":friends"],  
)
```

6. Managing Dependencies

6.1 Local Dependencies

```
cc_binary(  
    name = "app",  
    srcs = ["app.cpp"],  
    deps = [  
        "//src/lib:hello",          # Internal library  
        "//src/utls:string_utls",  # Another internal lib  
    ],  
)
```

6.2 External Dependencies via http_archive

```
# WORKSPACE  
load("@bazel_tools//tools/build_defs/repo:http.bzl", "http_  
  
# Google Test  
http_archive(  
    name = "com_google_googletest",  
    urls = ["https://github.com/google/googletest/archive/1  
    strip_prefix = "googletest-release-1.12.1",  
    sha256 = "24564e3b712d3eb30ac9a85d92f7d720f60cc0173730:  
)  
  
# Abseil C++  
http_archive(  
    name = "com_google_absl",  
    urls = ["https://github.com/abseil/abseil-cpp/archive/1  
    strip_prefix = "abseil-cpp-20230125.3",  
)
```

6.3 Using External Dependencies

```
cc_binary(  
    name = "advanced_app",
```

```

    srcs = ["advanced_app.cpp"],
    deps = [
        "@com_google_absl//absl/strings",
        "@com_google_absl//absl/container:flat_hash_map",
    ],
)

// advanced_app.cpp
#include "absl/strings/str_cat.h"
#include "absl/container/flat_hash_map.h"
#include <iostream>

int main() {
    absl::flat_hash_map<std::string, int> scores;
    scores["Alice"] = 95;
    scores["Bob"] = 87;

    for (const auto& [name, score] : scores) {
        std::cout << absl::StrCat(name, ": ", score) << std::endl;
    }
    return 0;
}

```

6.4 git_repository

```

# WORKSPACE
load("@bazel_tools//tools/build_defs/repo:git.bzl", "git_repository")

git_repository(
    name = "my_custom_lib",
    remote = "https://github.com/user/my-lib.git",
    tag = "v1.2.3",
)

```

7. Advanced C++ Features

7.1 Compiler Flags

```
cc_library(  
    name = "optimized_lib",  
    srcs = ["optimized.cpp"],  
    hdrs = ["optimized.h"],  
    copts = [  
        "-std=c++17",  
        "-O3",  
        "-Wall",  
        "-Wextra",  
        "-Werror",  
    ],  
)
```

7.2 Conditional Compilation

```
cc_library(  
    name = "platform_lib",  
    srcs = select({  
        "@platforms//os:linux": ["linux_impl.cpp"],  
        "@platforms//os:macos": ["macos_impl.cpp"],  
        "@platforms//os:windows": ["windows_impl.cpp"],  
        "//conditions:default": ["generic_impl.cpp"],  
    }),  
    hdrs = ["platform.h"],  
)
```

7.3 Custom Defines

```
cc_binary(  
    name = "debug_app",  
    srcs = ["app.cpp"],  
    defines = [  
        "DEBUG_MODE=1",  
    ],  
)
```

```
        "LOG_LEVEL=3",
    ],
    local_defines = [
        "INTERNAL_FLAG", # Only for this target
    ],
)
```

7.4 Include Paths

```
cc_library(
    name = "custom_include_lib",
    srcs = ["lib.cpp"],
    hdrs = ["lib.h"],
    includes = ["include"], # Add to include path
    strip_include_prefix = "src", # Strip prefix from header
)
```

7.5 Linking Options

```
cc_binary(
    name = "linked_app",
    srcs = ["app.cpp"],
    deps = [":mylib"],
    linkopts = [
        "-lpthread",
        "-ldl",
    ],
    linkstatic = True, # Static linking
)
```

7.6 Header-Only Libraries

```
cc_library(  
    name = "header_only_lib",  
    hdrs = ["template_lib.h"],  
    visibility = ["//visibility:public"],  
)
```

7.7 Precompiled Headers

```
cc_library(  
    name = "common_headers",  
    hdrs = ["pch.h"],  
)  
  
cc_binary(  
    name = "fast_compile_app",  
    srcs = ["app.cpp"],  
    deps = [":common_headers"],  
)
```

8. Testing with Bazel

8.1 Unit Tests

```
cc_test(  
    name = "math_test",  
    srcs = ["math_test.cpp"],  
    deps = [  
        "//src/lib:math",  
        "@com_google_googletest//:gtest_main",  
    ],  
    size = "small", # small, medium, large, enormous  
)
```

```
// math_test.cpp  
#include "src/lib/math.h"  
#include <gtest/gtest.h>  
  
TEST(MathTest, Addition) {  
    EXPECT_EQ(add(2, 3), 5);  
}  
  
TEST(MathTest, Multiplication) {  
    EXPECT_EQ(multiply(4, 5), 20);  
}  
  
class MathFixture : public ::testing::Test {  
protected:  
    void SetUp() override {  
        // Setup code  
    }  
    void TearDown() override {  
        // Cleanup code  
    }  
};  
  
TEST_F(MathFixture, ComplexOperation) {  
    // Test using fixture
```



```
}
```

8.2 Test Data

```
cc_test(  
    name = "file_processor_test",  
    srcs = ["file_processor_test.cpp"],  
    deps = [  
        "//src:file_processor",  
        "@com_google_googletest//:gtest_main",  
    ],  
    data = [  
        "testdata/input.txt",  
        "testdata/expected_output.txt",  
    ],  
)
```

```
// file_processor_test.cpp  
#include <fstream>  
#include <gtest/gtest.h>  
  
TEST(FileProcessorTest, ReadTestData) {  
    std::ifstream file("tests/testdata/input.txt");  
    ASSERT_TRUE(file.is_open());  
    // Test file processing  
}
```

8.3 Running Tests

```
# Run all tests  
bazel test //...  
  
# Run specific test  
bazel test //tests:math_test  
  
# Run with output  
bazel test //tests:math_test --test_output=all
```

```
# Run tests matching pattern
bazel test //tests:all --test_filter=*Addition*

# Run tests with specific tag
bazel test //... --test_tag_filters=integration
```

8.4 Test Suites

```
test_suite(
    name = "all_tests",
    tests = [
        ":math_test",
        ":string_test",
        ":file_test",
    ],
)
```

```
test_suite(
    name = "unit_tests",
    tags = ["unit"],
)
```

9. Build Optimization

9.1 .bazelrc Configuration

```
# .bazelrc
# Common flags for all commands
common --enable_platform_specific_config

# Build optimizations
build --jobs=auto
build --compilation_mode=opt
build --cxxopt=-std=c++17

# Debug configuration
build:debug --compilation_mode=dbg
build:debug --copt=-g
build:debug --strip=never

# Release configuration
build:release --compilation_mode=opt
build:release --copt=-O3
build:release --copt=-DNDEBUG
build:release --strip=always

# Test configuration
test --test_output=errors
test --test_summary=detailed

# Remote cache
build --remote_cache=https://cache.example.com
```

9.2 Compilation Modes

Mode	Flag	Description
fastbuild	<code>-C</code> <code>fastbuild</code>	Fast compilation, minimal optimization (default)

opt	-c opt	Optimized for performance
dbg	-c dbg	Debug symbols, no optimization

9.3 Caching

```
# Local disk cache
build --disk_cache=~/.cache/bazel

# Remote cache
build --remote_cache=grpc://cache.example.com:9092

# Remote execution
build --remote_executor=grpc://executor.example.com:9092
```

9.4 Incremental Builds

Tips for Faster Builds:

- Keep targets small and focused
- Minimize dependencies between targets
- Use header files properly to avoid unnecessary recompilation
- Enable remote caching for team builds

9.5 Build Analysis

```
# Analyze build performance
bazel build //src:main --profile=profile.json
bazel analyze-profile profile.json

# Show dependency graph
bazel query --output=graph //src:main > graph.dot
dot -Tpng graph.dot -o graph.png

# Show action graph
bazel aquery //src:main
```

10. Custom Toolchains

10.1 Defining a Toolchain

```
# toolchains/BUILD
load("@bazel_tools//tools/cpp:cc_toolchain_config.bzl", "cc_toolchain_config")

cc_toolchain_config(
    name = "my_toolchain_config",
    compiler = "gcc",
    cpu = "k8",
)

cc_toolchain(
    name = "my_cc_toolchain",
    all_files = ":compiler_files",
    compiler_files = ":compiler_files",
    dwp_files = ":empty",
    linker_files = ":linker_files",
    objcopy_files = ":empty",
    strip_files = ":empty",
    toolchain_config = ":my_toolchain_config",
)

toolchain(
    name = "my_toolchain",
    exec_compatible_with = [],
    target_compatible_with = [],
    toolchain = ":my_cc_toolchain",
    toolchain_type = "@bazel_tools//tools/cpp:toolchain_type"
)
```

10.2 Cross-Compilation

```
# Build for different platforms
bazel build //src:main --platforms=//platforms:linux_x86_64
bazel build //src:main --platforms=//platforms:macos_arm64
bazel build //src:main --platforms=//platforms:windows_x86_64
```

10.3 Custom Build Rules

```
# rules/proto.bzl
def _my_proto_library_impl(ctx):
    output = ctx.actions.declare_file(ctx.label.name + ".proto")

    ctx.actions.run(
        inputs = ctx.files.srcs,
        outputs = [output],
        executable = ctx.executable._protoc,
        arguments = [
            "--cpp_out=" + output.dirname,
            ctx.files.srcs[0].path,
        ],
    )

    return [DefaultInfo(files = depset([output]))]

my_proto_library = rule(
    implementation = _my_proto_library_impl,
    attrs = {
        "srcs": attr.label_list(allow_files = [".proto"]),
        "_protoc": attr.label(
            default = "@com_google_protobuf//:protoc",
            executable = True,
            cfg = "exec",
        ),
    },
)
```

11. Best Practices

11.1 Project Structure

```
project/
├── WORKSPACE
├── .bazelrc
├── .bazelversion
├── MODULE.bazel          # For Bzlmod (Bazel 6+)
├── src/
│   ├── BUILD
│   ├── main/
│   │   ├── BUILD
│   │   └── main.cpp
│   └── lib/
│       ├── BUILD
│       ├── feature_a/
│       │   ├── BUILD
│       │   ├── feature_a.h
│       │   └── feature_a.cpp
│       └── feature_b/
│           ├── BUILD
│           ├── feature_b.h
│           └── feature_b.cpp
├── tests/
│   ├── BUILD
│   ├── unit/
│   │   ├── BUILD
│   │   └── *_test.cpp
│   └── integration/
│       ├── BUILD
│       └── *_integration_test.cpp
└── third_party/
    └── BUILD
```

11.2 Target Naming Conventions

- Use lowercase with underscores: `my_library`
- Binary names should match their purpose: `main` , `server`
- Test names should end with `_test` : `math_test`
- Library names should be descriptive: `string_utils`

11.3 Dependency Management

 **Avoid These Common Mistakes:**